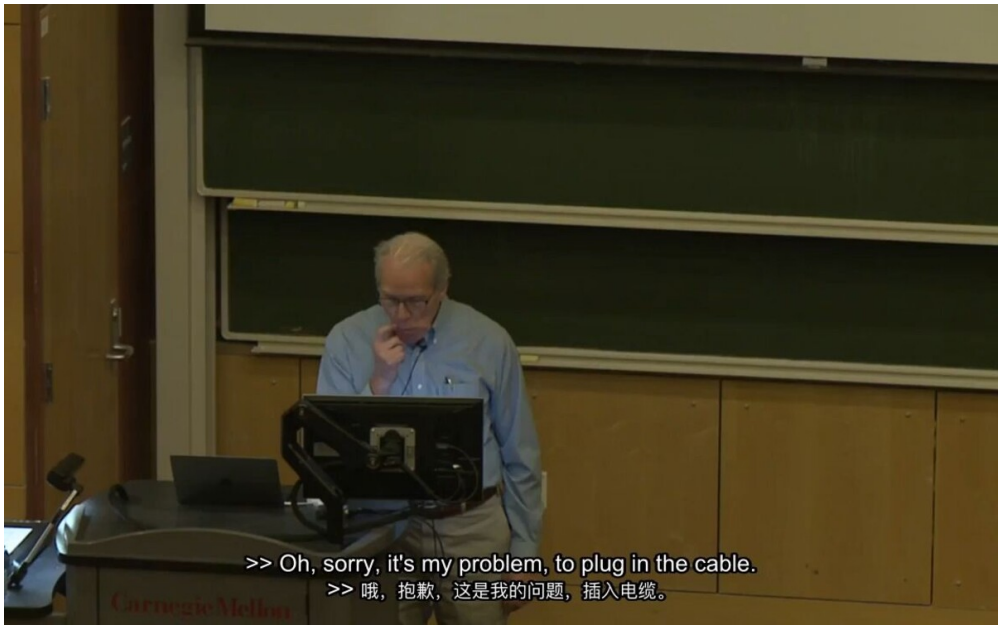
课程笔记

事务内存：从无锁的困境到声明式原子性

CMU 15-418/618 并行计算机架构与编程 · Lecture 22（Spring 2018）

lecture-to-notes

2026 年 10 月 29 日



视频作者/频道：CMU 15-418/618（B 站搬运：alexphil）
发布日期：2018 年春季学期
视频时长：1 小时 12 分 38 秒
视频链接：`https://www.bilibili.com/video/BV1nP411W7qr/`

目录

1 上一讲的遗留：ABA 与两种解法 **2**
1.1 ABA 的形态 . . . 2
1.2 解法一：计数器与双重比较交换 . . . 3
1.3 解法二：危险指针 . . . 3
1.4 两者是独立的解法 . . . 5
1.5 本章小结 . . . 5

2 为什么需要更高层的抽象 **6**
2.1 手工无锁的代价 . . . 6
2.2 抽象的目标 . . . 6
2.3 本章小结 . . . 7

3 事务：来自数据库的想法 **7**
3.1 经典例子 . . . 7
3.2 最直接的做法与它的不足 . . . 7
3.3 本章小结 . . . 8

4 声明式与命令式 **8**
4.1 两种抽象的差别 . . . 8
4.2 本章小结 . . . 9

5 事务的三个保证 **9**
5.1 原子性 . . . 10
5.2 隔离性 . . . 10
5.3 可串行化 . . . 10
5.4 本章小结 . . . 10

6 乐观执行与冲突检测 **11**
6.1 本章小结 . . . 11

7 乐观执行与冲突检测 **11**
7.1 理想情形 . . . 11
7.2 一个具体对比：哈希表与树 . . . 12
7.3 本章小结 . . . 13

8 总结与延伸 **13**

1　上一讲的遗留：ABA 与两种解法

上一讲讲了一个不涉及锁的无锁队列实现，并遇到了 ABA 问题。这一讲先把它收尾。

1.1　ABA 的形态

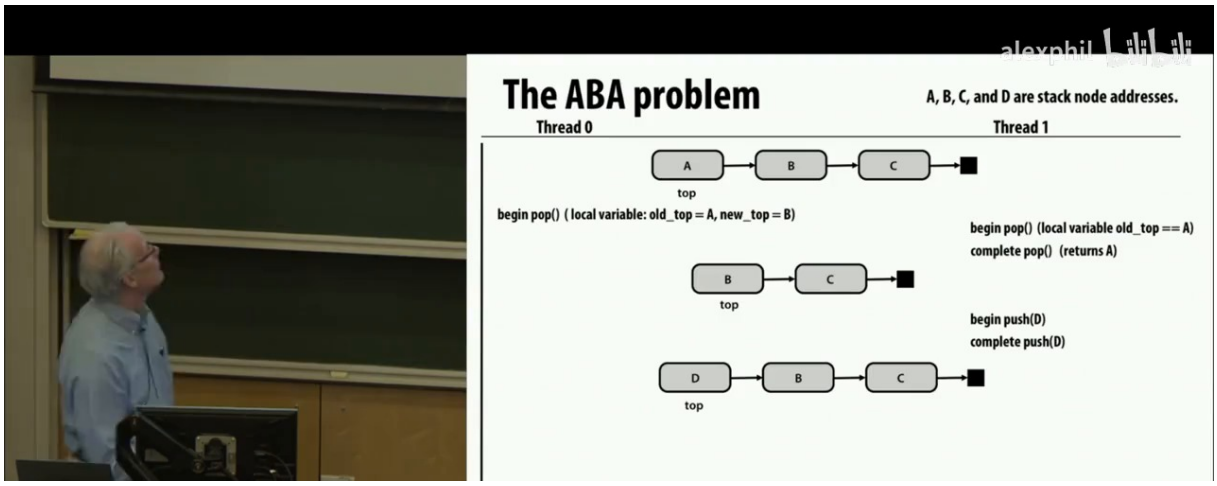


图 1: ABA 问题：指针值相同但所指内容已被改动[1]

讲者复述问题：你取得一个指向某物的指针，后来做比较并交换时看到同一个指针，于是认为它一定没变过。但在此期间，别的线程可能改动过这个指针或它所指向的东西，使其变得无效。

ABA 的本质是”值相等”不等于”历史相同”

比较并交换只能比较当前值，无法区分”一直没变”与”变过又变回来”。因此任何依赖 CAS 的判断都必须回答一个问题：这个值在被比较期间是否可能经历过变化再复原？若可能，就必须引入额外的历史信息（如计数器）或外部保护机制。

[1] 视频画面时间区间：00:01:30–00:01:45。若为裁剪图，时间区间仍对应原视频画面。

1.2 解法一：计数器与双重比较交换



图 2: 用计数器解决 ABA 的无锁栈[2]

讲者回顾上一讲提过的做法：给数据结构加一个额外计数器，用所谓的双重比较交换，使得你可以同时把这个元素从链表中摘除并原子地递增计数器。

于是你不仅能判断”数据是否还是原来那个”,还能判断”计数器是否成功递增”,从而避免 ABA。

他在这里做了一个诚实的更正：上次讲完后他说这里还有另一个问题，但课后有人指出这段代码其实完全正确，这是相当经典的实现。若去查 ABA 问题，会看到关于双重比较交换的讨论。讲者还提到一个边界情形：如果那个 pop 计数器只是 32 位整数，那么它会回绕。回绕之后，一个足够长的运行就可能让计数器回到旧值，从而重新暴露 ABA。因此在长时间运行的系统里，计数器的位宽必须与并发规模匹配。

1.3 解法二：危险指针

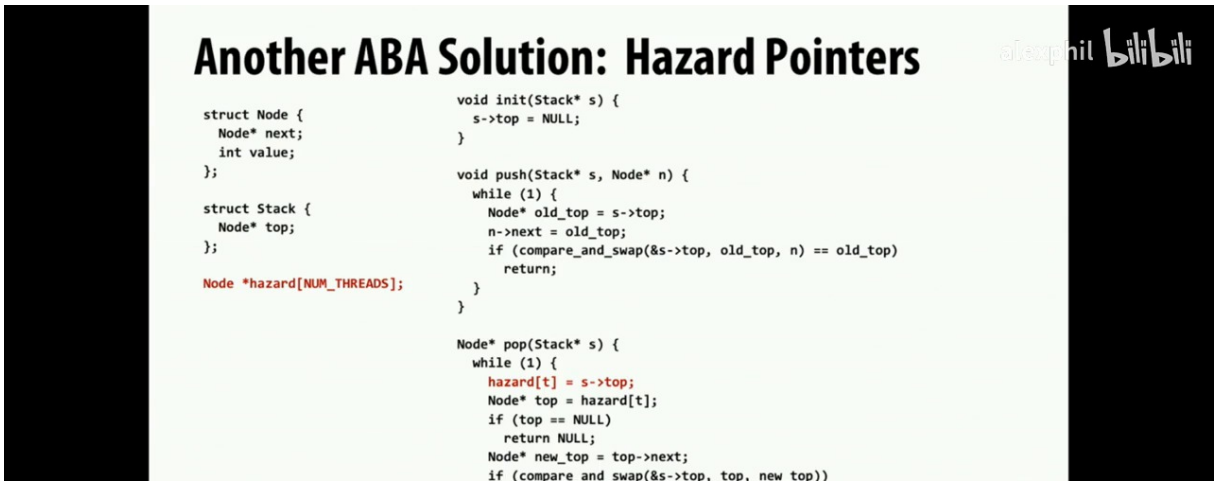


图 3: 另一种 ABA 解法：危险指针[3]

[2]视频画面时间区间：00:02:00–00:02:15。若为裁剪图，时间区间仍对应原视频画面。

[3]视频画面时间区间：00:05:30–00:05:45。若为裁剪图，时间区间仍对应原视频画面。

讲者接着介绍危险指针，说它其实是解决 ABA 的另一种完全不同的思路，而且是真正被使用的技术。

它的主要优势是不需要双重比较交换，普通 CAS 就够。

做法是：有一个按线程组织的指针数组，每个线程可以列出它正在持有的一个项，表示”我正在用这个，别人不要动它”。

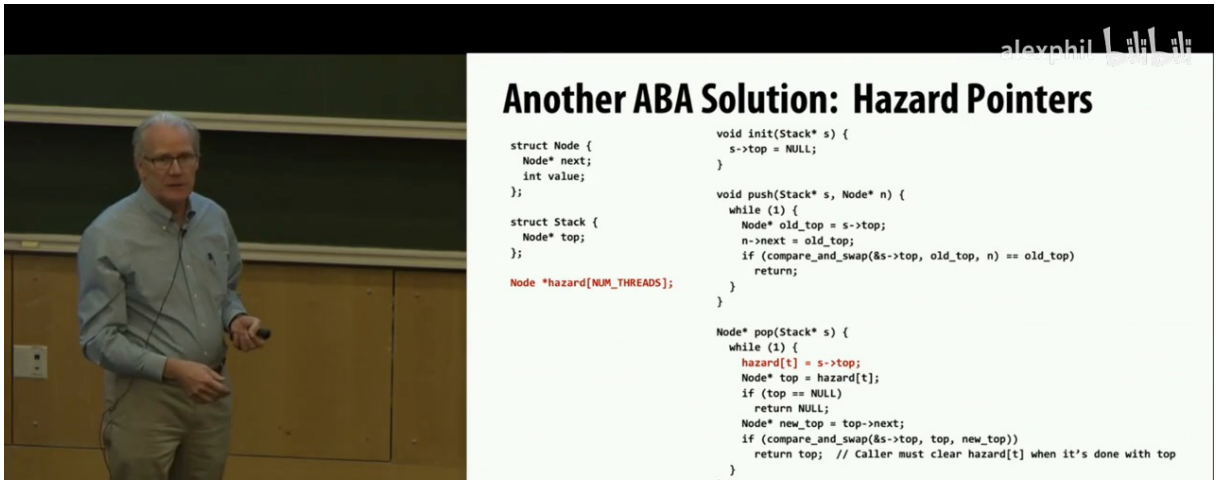


图 4: 危险指针续：调用者取到栈顶后保持标记为真[4]

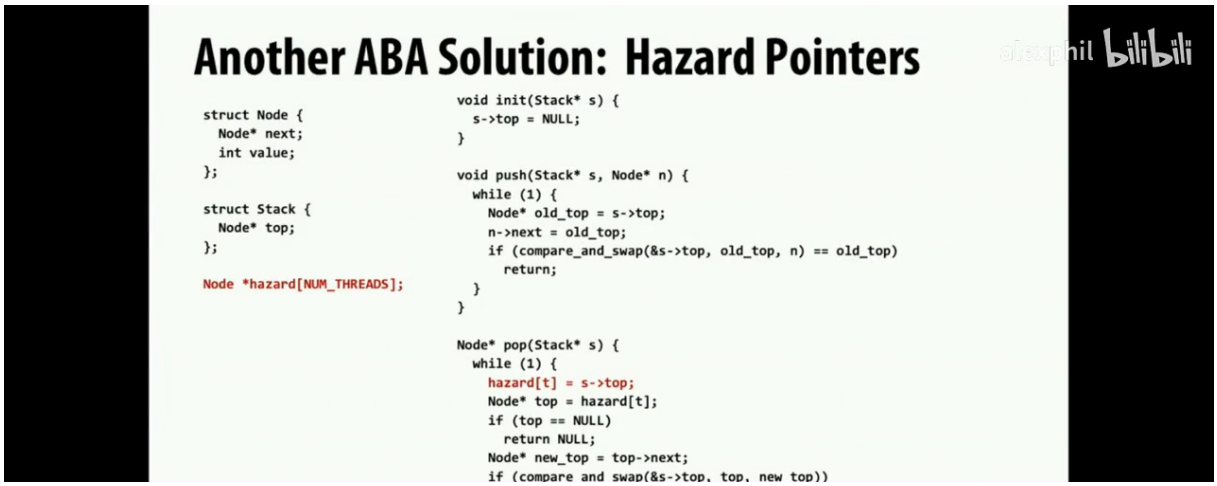


图 5: 危险指针续：回收节点前必须确认无人标记[5]

即使操作完成后，调用者仍会得到一个栈顶值；只要它还在引用那个栈顶，就应把危险标志保持为真。

一般规则是：任何时候你想回收或释放一个节点，只要还有线程把它标记为自己的危险指针，就不能这么做。

[4]视频画面时间区间：00:07:00–00:07:15。若为裁剪图，时间区间仍对应原视频画面。

[5]视频画面时间区间：00:07:30–00:07:45。若为裁剪图，时间区间仍对应原视频画面。

危险指针把”谁在用”变成可查询的全局状态

双重 CAS 用版本号记录”历史变过没有”；危险指针用共享标记记录”现在谁在用”。前者解决 ABA，后者同时解决”引用已释放内存”。因此两者不是同一问题的两种写法，而是针对不同侧面的机制。

1.4 两者是独立的解法

讲者澄清一个上次没说清的点：上次他讲危险指针时把它说成”要在计数器之外额外做的事”，但实际上它们是两种独立的实现方式。

危险指针不需要特殊指令，但需要某种更全局的控制与状态跟踪。

他还说危险指针在”不只是字级别、而是想对整个对象做原子操作”时也有用，因此它更多出现在高层协议里。

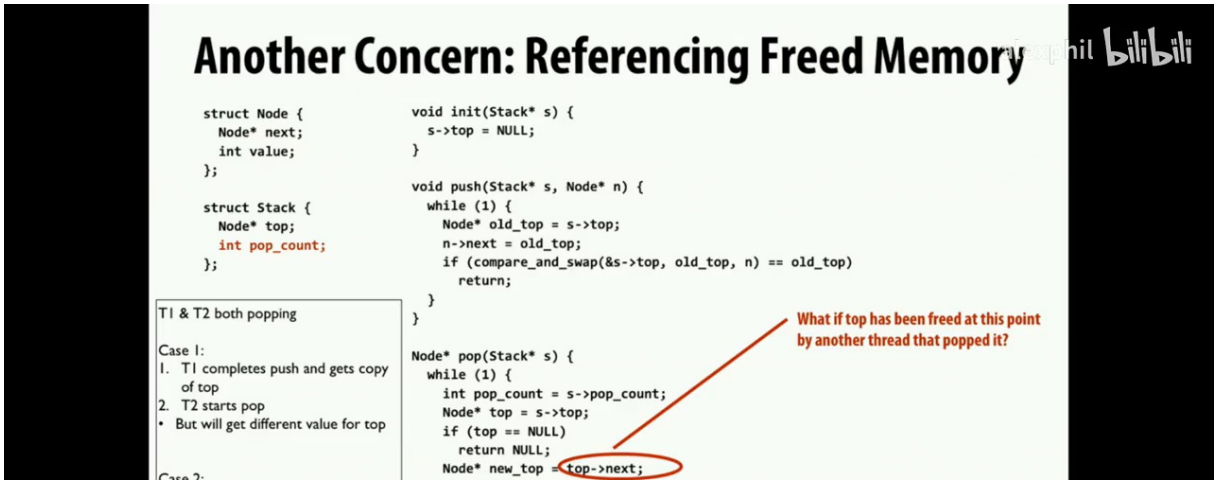


图 6: 另一个问题：引用已释放的内存[6]

1.5 本章小结

ABA 指指针值相同而所指内容已被改动，本质是”值相等”不等于”历史相同”。两种独立解法是：加计数器并用双重比较交换，可同时摘除元素与递增计数；或用危险指针，让每个线程标记正在持有的节点，回收前须确认无人标记。后者不需要特殊指令，但需要全局协调，也适用于整对象的原子操作。

[6]视频画面时间区间：00:03:00–00:03:15。若为裁剪图，时间区间仍对应原视频画面。

2 为什么需要更高层的抽象

2.1 手工无锁的代价

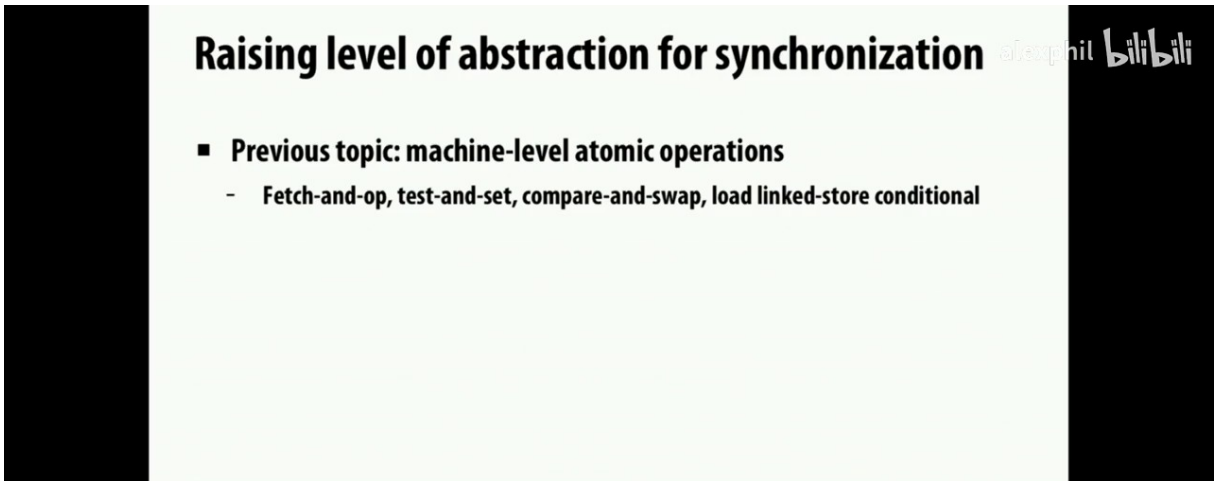


图 7: 提升同步的抽象层次：从机器级原子操作往上走[7]

讲者回顾上一段内容：我们一直在各种锁与技巧之间打转，或者像这个例子一样，直接停在数据结构层面思考如何无锁地实现。

他的评价很直接：这很枯燥，风险很高。

风险具体在哪？你可能觉得它工作正常，在某种处理器上测过；然后换一个型号，比如从 x86 换到 ARM，突然就不工作了。

2.2 抽象的目标

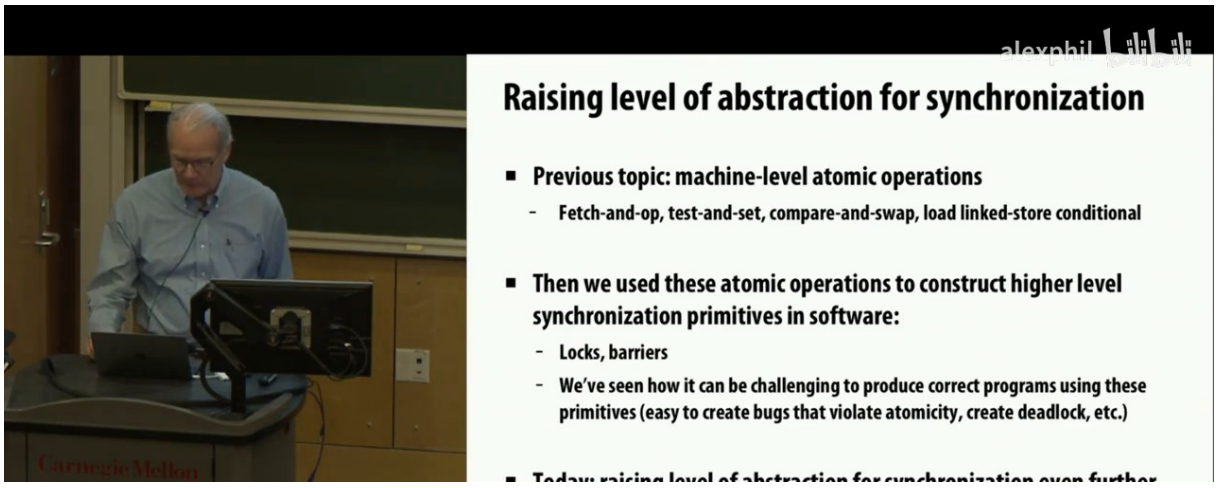


图 8: 提升抽象层次续：由编译器与硬件的组合决定实现[8]

因此今天想做的是：能否提升到更高的抽象层，表达一些我们想实现的高层原则，然后用编译器与硬件的某种组合让它们工作，并由系统去判断在具体情形下什么是最好的做法。

[7] 视频画面时间区间：00:10:00–00:10:15。若为裁剪图，时间区间仍对应原视频画面。

[8] 视频画面时间区间：00:11:00–00:11:15。若为裁剪图，时间区间仍对应原视频画面。

讲者加了一句玩笑：这除了会减少系统程序员的工作机会之外，一切都很好——而这目前似乎不是问题。

抽象提升的代价是失去控制权

当编译器与硬件替你决定”如何实现原子性”时，你也失去了对开销、延迟、回退策略的控制。因此在性能敏感的场合，高层抽象可能不够用。这也是事务内存在实践中长期处于”可用但不总是更快”状态的原因。

2.3 本章小结

手工实现无锁结构枯燥且风险高：在一种处理器上正常不代表换架构后仍正常。因此目标是提升抽象层，由编译器与硬件的组合决定实现。代价是失去对开销与策略的控制，这也是高层抽象在实践中不一定更快的原因。

3 事务：来自数据库的想法

3.1 经典例子

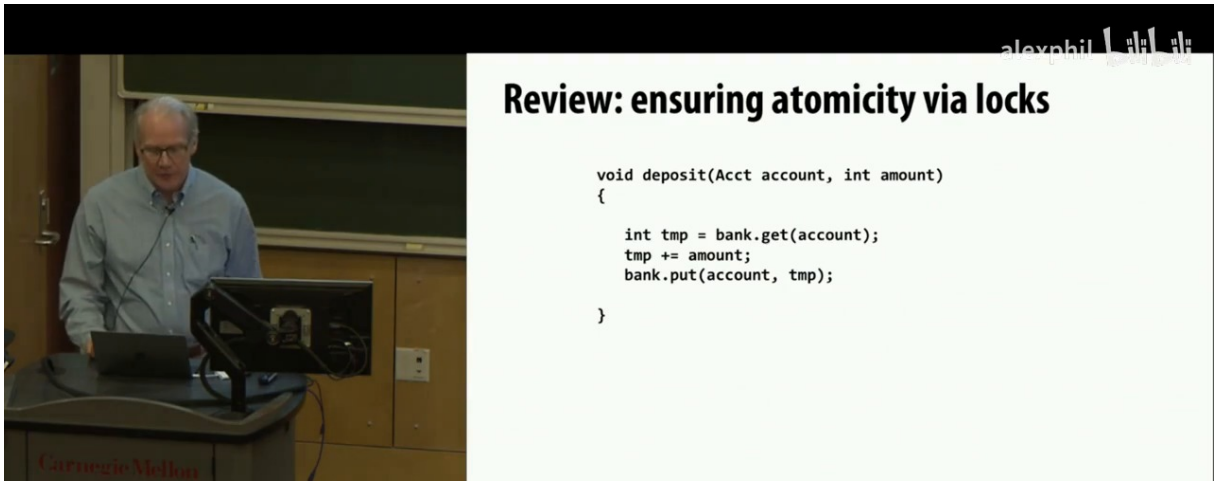


图 9: 回顾：用锁保证原子性[9]

讲者说明事务的想法来自数据库社区，他们已经研究了几十年。

经典例子是银行账户：如果要把钱从账户 A 转到账户 B，我希望这件事以一种不会留下不一致状态的方式完成——比如钱从 A 扣了却没能存进 B。

这就是原子性：要么完全做完，要么完全不做。若某个事务失败，我希望它就像从未发生过一样。

他补充说，即使只是单纯往账户里存款（不涉及转账），也希望它要么成功且计数确实增加了，要么失败，并且事后我能知道它没有发生。

3.2 最直接的做法与它的不足

讲者说起点就是给它套一把大锁。这至少能保证不存在竞争条件导致账户被错误递增。

[9] 视频画面时间区间：00:12:30–00:12:45。若为裁剪图，时间区间仍对应原视频画面。

但这不是完整的方案。他真正想要的是类似在 OpenMP 里见过的那种：直接声明某段代码是原子的，然后由编译器、操作系统与硬件去想办法实现。

3.3 本章小结

事务思想来自数据库，已有数十年历史。经典例子是银行转账：原子性要求要么全做完要么全不做，失败则如同从未发生。最直接的做法是套一把大锁，它能避免竞争但不是完整方案——理想是直接声明某段代码为原子。

4 声明式与命令式

4.1 两种抽象的差别

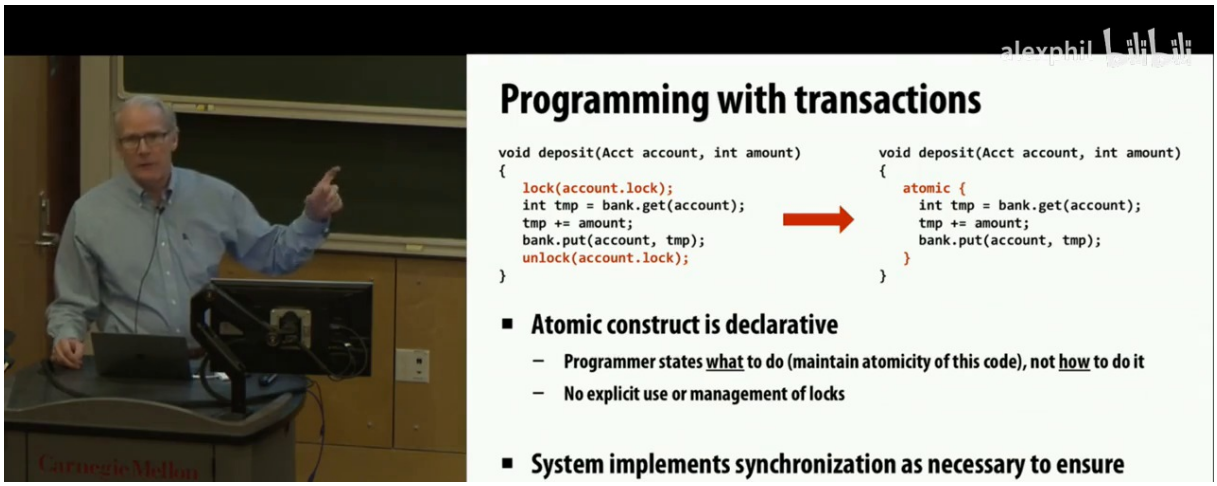


图 10: 用事务来编程：声明一段原子操作[10]



图 11: 声明式与命令式抽象的区别[11]

[10] 视频画面时间区间：00:13:30–00:13:45。若为裁剪图，时间区间仍对应原视频画面。
[11] 视频画面时间区间：00:14:30–00:14:45。若为裁剪图，时间区间仍对应原视频画面。

讲者给出定义。声明式是指：我只给出想做什么的意图。命令式是指：我要按下面的方式做——设置一把锁、做一些操作、释放锁。

事务内存的想法就是为内存系统提供这种原子能力，靠硬件、软件与编译器的某种组合来实现。

$$\text{声明式}:\ \text{atomic}\{\ \text{operations}\ \} \quad \longrightarrow \quad \text{命令式}:\ \text{lock};\ \text{operations};\ \text{unlock}$$

- 左侧只说明”这段要原子”，具体机制由系统选择；
- 右侧显式指定用锁，实现细节暴露给程序员；
- 左侧可以随硬件演进更换实现而不改代码，右侧不能。

声明式的价值在于把”怎么做”留给系统

同一段声明式代码，在不同硬件上可以被编译成锁、硬件事务、或软件事务。这意味着程序的可移植性从”语法层”提升到”语义层”。而命令式代码把机制写死，换平台时必须重写。

4.2 本章小结

声明式只给出意图，命令式给出做法。事务内存为内存系统提供原子能力，用硬件、软件与编译器组合实现。声明式的价值在于同一份代码可随硬件演进更换实现而不必重写。

5 事务的三个保证

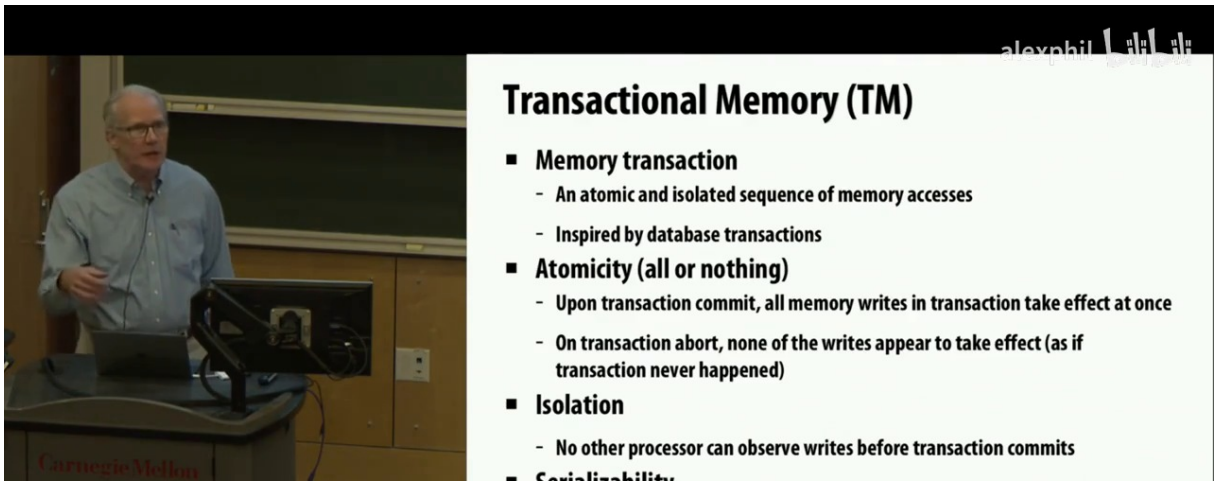


图 12: 事务内存：一段原子且隔离的内存访问序列[12]

[12] 视频画面时间区间：00:15:00–00:15:15。若为裁剪图，时间区间仍对应原视频画面。

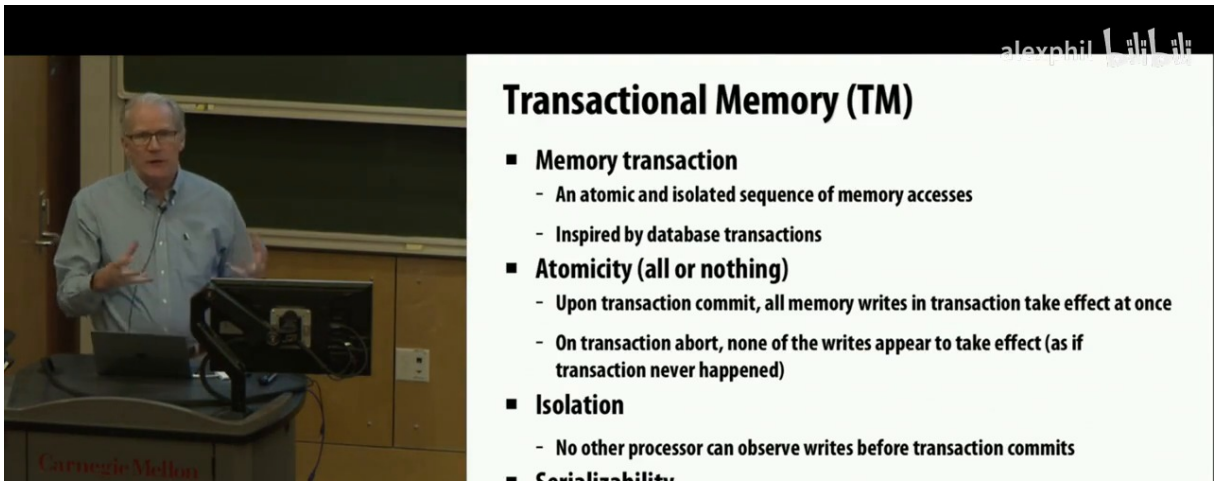


图 13: 事务内存的特性：原子性、隔离性与可串行化[13]

讲者列出希望具备的几个特性，它们都超出此前看过的普通同步。

5.1 原子性

第一个是原子性：要么全做要么全不做。括号里的每一个步骤都要发生，或者一个都不发生，不留任何残余。

比如作为转账的一部分，我绝不会出现” 从一个账户取出了钱但没能存进另一个账户”。

5.2 隔离性

第二个比此前看过的更强，叫隔离性：当事务进行时，如果有其他观察者对这些状态有只读访问权，他们不应看到任何中间状态。

也就是说，若他们在任意时刻查看银行账户，不应看到转账过程中某一个账户被扣减而另一个尚未增加的瞬间。对外部观察者而言，即使他们没有写权限，也应看到” 突然一下，两个账户的值一起变了”。

5.3 可串行化

第三个是可串行化，此前讨论过：我不关心事务在时钟意义上的确切顺序，但我关心它们是否可串行化——我可以把这段执行重放成某个序列，使它看起来像纯顺序发生的一样。

三个保证是递进关系而非并列

原子性保证” 不半途而废”，隔离性保证” 外人看不到中间态”，可串行化保证” 整体等价于某个串行顺序”。前两者约束单个事务的可见性，第三者约束多个事务之间的等价性。缺任何一条，程序员的推理都会失效。

5.4 本章小结

事务的三个保证是：原子性（全做或全不做，不留残余）、隔离性（外部观察者看不到中间状态，即使只读也不行）、可串行化（不关心时钟顺序，但可重放为某个串行序列）。三者递进：分别约束

[13] 视频画面时间区间：00:16:30–00:16:45。若为裁剪图，时间区间仍对应原视频画面。

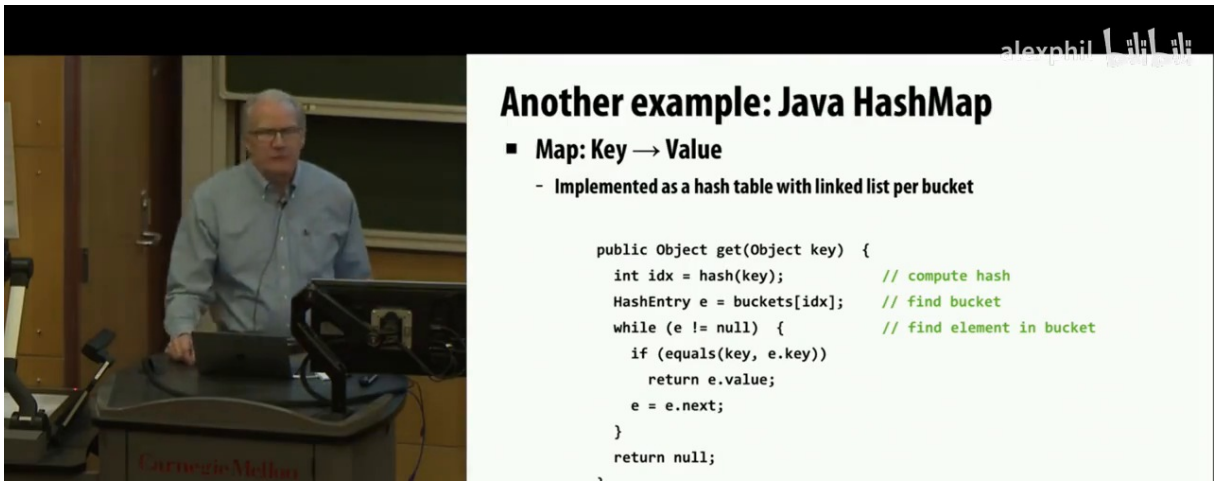


图 14: 事务内存的实现思路：按串行版本执行并检测冲突[14]

单事务的完整性、单事务的可见性、多事务间的等价性。

6 乐观执行与冲突检测

6.1 本章小结

事务的三个保证是：原子性（全做或全不做）、隔离性（外部看不到中间状态）、可串行化（可重放为串行序列）。三者递进：分别约束单事务的完整性、单事务的可见性、多事务间的等价性。

有了保证，接下来的问题是系统如何实现它们。

7 乐观执行与冲突检测

7.1 理想情形

讲者描述理想情形：我们希望能直接说”就原子地做这件事”，然后由系统去想办法实现。

而且可能不必靠细粒度锁，而是采用另一种思路：先假设没问题，就像串行版本那样执行，同时检查途中是否发生了冲突。

冲突的判据可以写成读写集合的交集：

$$\text{conflict} \iff (R_i \cap W_j) \cup (W_i \cap R_j) \cup (W_i \cap W_j) \neq \varnothing$$

- R_i, W_i：事务 i 读过的地址集合与写过的地址集合；
- 任意两项非空即表示两个事务之间存在冲突；
- 检测到冲突时，至少有一个事务必须回滚重试。

若两个事务的读写集合完全不相交，则它们可以并行完成而不必协调。

[14] 视频画面时间区间：00:25:30–00:25:45。若为裁剪图，时间区间仍对应原视频画面。

乐观执行把”预防”换成”检测”

细粒度锁是预防式的：进入前先取得权限，因此冲突不会发生。乐观执行是检测式的：先做，再检查是否被别人干扰过，若被干扰则回滚重试。前者在低冲突时开销偏高（每次都要加锁），后者在低冲突时几乎无开销，但在高冲突时会反复回滚。

重试的期望次数随冲突概率上升而急剧增加：

$$E[\text{retries}] = \frac{1}{1 - p_{\text{conflict}}}$$

- p_{conflict}：一次尝试中与并发事务冲突的概率；
- 当 p_{conflict} 接近 1 时，期望重试次数趋于无穷；
- 因此乐观执行在高冲突下会失效，必须辅以退避或切换到悲观模式。

7.2 一个具体对比：哈希表与树

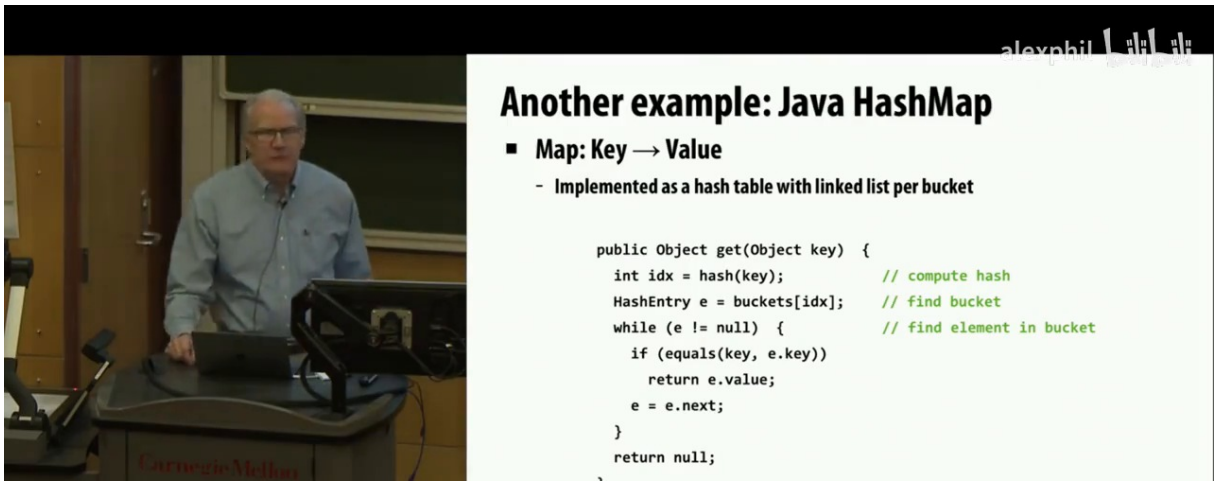


图 15: 具体例子：Java HashMap 的桶式实现[15]

讲者用哈希表与树的对比说明为什么这个区别重要。

哈希表可以直接做到 N 路分裂：计算哈希函数本身不需要锁，因此可以立刻跳到许多互不相关的链表上并行操作。

而树结构不一样：所有人都必须先穿过根，再穿过头几层。若树足够大，之后可以散开并行，但他们在根处是串行化的。

数据结构的形状决定了并行的天然上限

哈希表在入口处就把工作分成独立分支，因此并行度随桶数增长；树的入口是单一节点，因此并行度被根的限制压住。这不是锁实现得好不好的问题，而是结构本身的性质。因此”选什么数据结构”在并行场景下比”怎么加锁”更根本。

[15] 视频画面时间区间：00:28:00–00:28:15。若为裁剪图，时间区间仍对应原视频画面。

7.3 本章小结

理想情形是声明” 原子地做” 并由系统实现，且可以不走细粒度锁，而是乐观执行：按串行版本做，同时检测冲突，有冲突则回滚。哈希表因哈希函数无需加锁而可 N 路分裂；树的入口单一，所有人须先穿过根，因此天然串行化。这说明数据结构的形状决定了并行度的上限。

8 总结与延伸

讲者在结尾承认事务方法仍有许多已讨论过的现实困难，但能在更高层次表达这件事本身非常有价值。

把全讲串起来，这是一条从” 手工无锁的泥潭” 走向” 声明式原子性” 的路径。上一讲的无锁队列遇到 ABA：指针值相同而所指内容已被改动，本质是” 值相等” 不等于” 历史相同”。两种独立解法是加计数器配双重比较交换，或使用危险指针——后者让每线程标记正在持有的节点，回收前须确认无人标记，不需要特殊指令但需要全局协调，也适用于整对象的原子操作。手工实现无锁结构枯燥且风险高：在一种处理器上正常不代表换架构后仍正常。因此目标是提升抽象层，由编译器与硬件决定实现，代价是失去对开销与策略的控制。事务思想来自数据库：原子性要求全做或全不做、失败如同未发生；最直接做法是套一把大锁，但那不是完整方案。声明式只给出意图，命令式给出做法，而声明式的价值在于同一份代码可随硬件演进更换实现。事务的三个保证是原子性、隔离性与可串行化，它们递进而非并列。实现上可以采用乐观执行：按串行版本做，同时检测冲突，有冲突则回滚。哈希表因哈希函数无需加锁而可 N 路分裂，树的入口单一因此天然串行化——数据结构的形状决定了并行度的上限。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 三个保证是递进的：原子性约束单事务的完整性，隔离性约束其对外可见性，可串行化约束多事务之间的等价性。只满足前两者而违反第三者，程序仍可能得到无法用任何串行顺序解释的结果。
- 危险指针与双重 CAS 解决的是不同侧面。前者回答” 现在谁在用这个节点”，后者回答” 这个值的历史是否变过”。因此它们可以互补，但不是同一问题的两种等价写法。
- 乐观执行的适用性取决于冲突率。低冲突时它几乎无开销，高冲突时会反复回滚，实际吞吐可能低于细粒度锁。因此” 事务比锁好” 不成立，只有” 在特定冲突率下更好”。
- 数据结构的形状比同步机制更根本。哈希表天然可分裂，树天然在根处串行。这一点在选择数据结构时就已决定，后续任何同步优化都无法突破。

这堂课的边界在于它是概念导论：讲者明确说事务方法仍有大量现实困难，本讲给出的是动机、保证与实现思路，不含具体的硬件事务指令或软件事务库的使用细节。

两个可以直接从本讲数据提出的下一步问题：其一，乐观执行在高冲突下会反复回滚，那么是否存在一种自适应策略，能在运行时检测冲突率并切换到悲观（加锁）模式？其二，可串行化允许重排为任意串行顺序，那么对于依赖特定顺序的算法，事务内存是否提供了表达顺序约束的手段？